

HTS MASTER CLASSIFIER

NLP-Based Product Classification for International Trade

Milestone 3 — Model Evaluation

IE7500 | Data Analytics Engineering | 2026

Group 4

Dario Garza · Hannah Lin · Clemence Umutoni · Dwayne Ford

1. Define Objectives

This project develops an NLP-based multiclass classification system that predicts Harmonized Tariff Schedule (HTS) codes from unstructured product descriptions found in trade documents. The core NLP task is supervised multiclass text classification: the input is a natural-language product description and the output is a predicted 4-digit HTS heading. Supporting tasks include text preprocessing, HTS code extraction, label encoding, TF-IDF feature extraction, model training, model evaluation, and inference.

2. Benchmarking

The benchmark compares four completed models: Logistic Regression, Linear SVM, Naive Bayes, and FFNN.

Model	Method	Advantages	Disadvantages
Model 1	Logistic Regression + TF-IDF	Fast, interpretable, strong baseline	Lower performance than SVM and FFNN
Model 2	Linear SVM + TF-IDF	Strong performance with sparse text features	Less naturally probabilistic; may need calibration
Model 3	Naive Bayes + TF-IDF	Very fast, simple, useful baseline	Weakest performance due to class imbalance and overlapping vocabulary

Model	Method	Advantages	Disadvantages
Model 4	FFNN + TF-IDF	Captures nonlinear patterns and performs well	More computationally expensive and requires more tuning

4. Performance Metrics

4.1 Choose Appropriate Metrics

Because HTS heading prediction is a supervised multiclass classification problem, we selected classification-appropriate metrics rather than language-generation metrics such as BLEU or ROUGE. Four metrics were tracked for every model:

- **Accuracy** — the proportion of product descriptions assigned the correct 4-digit HTS heading out of all test samples. This gives an overall, easy-to-communicate measure of pipeline performance.
- **Weighted Precision** — precision computed per class and averaged, weighted by the number of true instances of each class. This accounts for the 258-class imbalance in the CROSS rulings dataset.
- **Weighted Recall** — recall computed per class and averaged using the same class-weighting scheme, capturing how well the model finds all instances of each heading, including less-frequent ones.
- **Weighted F1-score** — the harmonic mean of weighted precision and recall, used as the primary model-selection metric because it balances false positives and false negatives across an imbalanced label set.

Precision, Recall, and F1 were computed with weighted averaging so that common HTS headings which are also common in real-world trade volume contribute proportionally more to the reported score, while still penalizing poor performance on rare classes.

4.2 Evaluation Process

All four models were trained and evaluated on the same 80/20 stratified train-test split of the 17,378-record, 258-class dataset (about 13,902 training records and 3,476 held-out test records), so any differences in performance come from the models themselves and not from different data. Each model ran once against this shared test set using the same 27,534-feature TF-IDF matrix, and Accuracy, weighted Precision, weighted Recall, and weighted F1 were calculated with scikit-learn's built-in metrics.

Since a single split could just get lucky, the winning Linear SVM model was retrained and re-tested with five different random seeds (42, 1, 7, 123, 2026). Accuracy stayed between 67.4% and 68.5%, and weighted F1 stayed between 66.8% and 68.0%, a mean of 67.9% accuracy and 67.5% F1, both with less than half a percentage point of spread. SVM performs around the same regardless of which specific rows end up in the test set.

4.3 Comparison to Baselines

Logistic Regression and Naive Bayes serve as the project's baseline models: Logistic Regression as a fast, linear, interpretable baseline, and Naive Bayes as a simple probabilistic baseline. Linear SVM and the FFNN represent the more capable approaches being evaluated against those baselines.

- Linear SVM vs. Naive Bayes baseline: +39.2 percentage points in accuracy (68.1% vs. 28.9%) and +44.4 points in weighted F1 (67.6% vs. 23.2%) — the largest margin in the comparison.

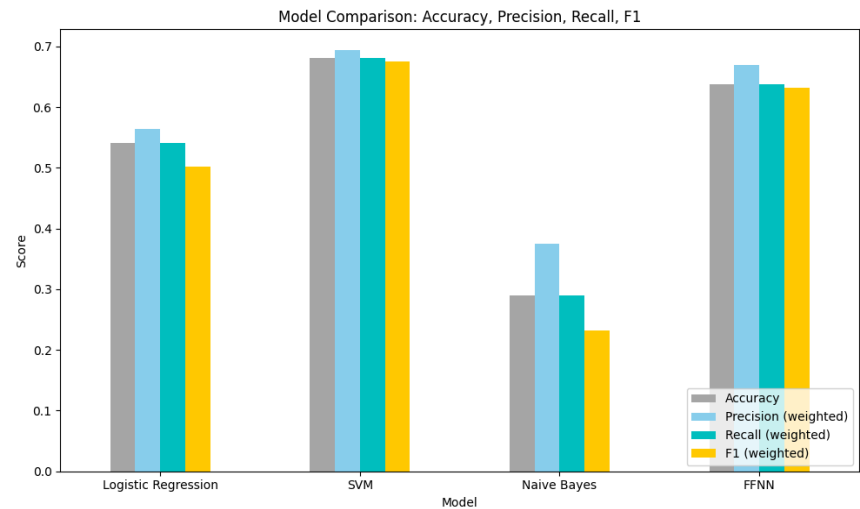
- Linear SVM vs. Logistic Regression baseline: +14.0 points in accuracy (68.1% vs. 54.1%) and +17.5 points in weighted F1 (67.6% vs. 50.1%), a roughly 26% relative improvement in accuracy.
- FFNN vs. Logistic Regression baseline: +9.7 points in accuracy and +13.2 points in weighted F1, showing that a nonlinear model adds value over a linear baseline even without embeddings, though it still trails SVM.

5. Results of Model Evaluation

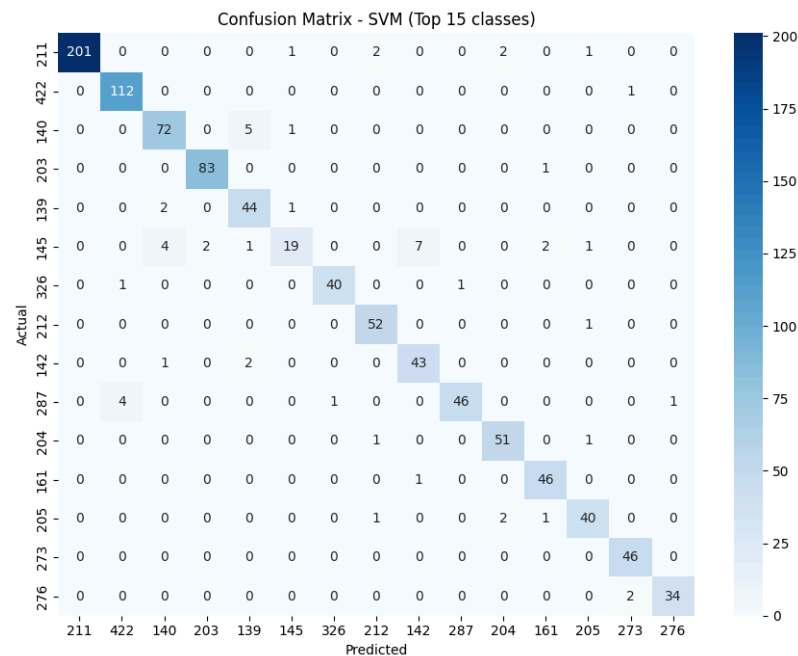
Model	Accuracy	Weighted Precision	Weighted Recall	Weighted F1
Logistic Regression	54.1%	56.4%	54.1%	50.1%
Linear SVM	68.1%	69.4%	68.1%	67.6%
Naive Bayes	28.9%	37.5%	28.9%	23.2%
FFNN	63.8%	66.9%	63.8%	63.3%

The team evaluated four completed models using the same train-test split and TF-IDF feature representation, allowing a consistent comparison across accuracy, weighted precision, weighted recall, and weighted F1-score. Overall, SVM is the best-performing model, achieving the highest accuracy and weighted recall at 68.1%, weighted precision at 69.4%, and weighted F1-score at 67.6%. This suggests SVM is well suited to HTS classification because product descriptions contain many keyword-based signals, and SVM performs effectively with high-dimensional sparse TF-IDF features.

The FFNN is the second-best model, with a weighted F1-score of 63.3%. Although its training loss decreased over 100 epochs, it did not generalize as well as SVM on the test set. Logistic Regression remains a useful fast, interpretable baseline at 50.1% weighted F1. Naive Bayes performed weakest, at 23.2% weighted F1, likely due to class imbalance, the large number of HTS classes, and overlapping vocabulary across headings.



Visualization 1: Model Comparison — Accuracy, Precision, Recall, F1



Visualization 2: Confusion Matrix — Linear SVM (Top 15 Classes)

6. Analysis and Insights

6.1 Interpret the Results

Linear SVM's 68.1% accuracy and 67.6% weighted F1 are well above the 28.9% accuracy of the Naive Bayes baseline, and also above the roughly 0.4% accuracy random guessing would produce across 258 classes. The five-seed consistency check also shows this performance holds steady across different train-test splits, so the model is picking up a real pattern between product-description wording and HTS headings rather than fitting to one particular split. However, roughly one in three test descriptions is still misrouted to the wrong heading, which in a production trade-compliance setting would need a review step.

6.2 Identify Strengths

Linear SVM does well with the high-dimensional, sparse TF-IDF space (27,534 features). This fits with how linear classifiers like this tend to perform on bag-of-words text. It also uses balanced class weighting (`class_weight='balanced'`), which directly deals with the class imbalance in this 258-class dataset. None of the other three models do this, and it's likely a big part of why SVM comes out ahead. Looking at the confusion matrix, most of the top 15 classes are rarely confused with anything else, suggesting the model is not just guessing on the common categories.

The model also demonstrated consistent performance across multiple random seeds, indicating good stability and robustness. Compared with the FFNN, it achieves competitive performance while requiring less computational effort, making it a practical choice for real-world applications. In addition, the end-to-end pipeline is fully reproducible, making future retraining, evaluation, and deployment straightforward.

6.3 Discuss Limitations

- Bias in data: Even after filtering, class sizes range from about 10 to over 1,000 examples, so the model sees far more of the common headings than the rare ones. The dataset itself may also skew toward unusual or disputed classification cases, since it's built from customs rulings rather than everyday trade descriptions.
- Overfitting: The FFNN's training loss dropped near zero (as low as 0.0003) by epoch 50–60, while test F1 leveled off at 63.3%. That gap suggests the model memorized training data more than it learned generalizable patterns, 0.3 dropout wasn't enough to prevent it.
- Generalization to unseen examples: SVM held up well across different splits of the same data (Section 4.2), but TF-IDF only matches exact words. A description phrased differently than training data, a synonym, unusual product name, or different writing style, could still get misclassified.

6.4 Suggest Improvements

- Hyperparameter tuning: All of the four models ran on default settings or values we picked without testing alternatives. There's room to improve by tuning SVM's C value or the FFNN's hidden layer size and learning rate.
- Some HTS headings only have 10-15 examples after filtering, which isn't much for the model to learn from. Either generating more examples for those rare classes or applying the same class-weighting SVM already uses to the other models might help even out performance.
- Since TF-IDF only catches exact word matches, trying a model that understands context and meaning like a fine-tuned BERT model, could help with cases where two descriptions use different wording for the same kind of product.

7. Error Analysis

7.1 Identify Common Mistakes

We looked at all 1,108 misclassified test descriptions and split them into two groups: cases where the predicted heading was at least in the same general product category as the correct one (same first two digits), and cases where it was a completely different category. 628 of the 1,108 (57%) were near-misses in the same category, and 480 (43%) were in an unrelated category.

For the near-miss cases, we found things like a boys' vest getting classified under jackets instead of pullovers, or one clothing subheading instead of a very similar one. While the model is picking up the right general area, it cannot tell two closely related headings apart, since they use a lot of the same words.

There was also a pattern for products described as bundles or multi-part items instead of one single product like a "golfer's water bottle assortment" that includes a bottle, tees, and a towel, or a strap/harness/waistpad "assembly." When a description lists several different components, the model guesses based on the most common instead of finding the actual primary product.

7.2 Analyze Failure Cases

Some examples of failure cases include: "High purity copper plated discs used in semiconductor sputtering tools" should be 7409 (copper plates), but the model said 8486 (semiconductor machinery). Another example is "various gas stove parts made of brass and steel". This was supposed to be 8482, the model said 7321, which is the code for actual stoves. With the word "Stove" being in the text, the model might have matched on that instead.

We also noticed the exact same long description for a multi-function tool with a motor, flexible shaft, rotary handpiece, etc. shows up twice with two different correct answers: 8204 and 8466 and both

times the model predicted 8205. Suggesting either this product is hard to classify between two headings, or there's an inconsistency in how it got labeled in the original data.